

Review Pembelajaran & Implementasi

Handling Dynamic Element dan Parallel Execution

6	0	0	100%
tests	failures	skipped	successful

Tujuan review. Dokumen ini merangkum pembelajaran, hasil implementasi, dan review singkat praktik handling dynamic element serta parallel execution pada automation test berbasis Gradle. Fokus utama adalah kestabilan locator/wait, efisiensi waktu eksekusi, dan kualitas hasil test report.

1. Ringkasan hasil eksekusi

Berdasarkan Gradle Test Report pada screenshot yang diberikan, **seluruh test berhasil dieksekusi tanpa failure maupun skipped test**. Total durasi run adalah sekitar 1 menit 15,18 detik. Detail child test menunjukkan tiga kelompok test: check-out, home page, dan login page.

Test suite / child	Tests	Failures	Skipped	Duration
test.CheckOutPageTest	1	0	0	1m5.42s
test.HomePageTest	4	0	0	1m12.36s
test.LoginPageTest	1	0	0	48.487s

Review hasil

Nilai 100% successful menunjukkan implementasi saat ini sudah menghasilkan eksekusi yang stabil pada run yang dilaporkan. Namun, kestabilan satu run belum otomatis membuktikan bahwa test bebas flaky; reliability perlu divalidasi dengan repeated run dan eksekusi paralel.

2. Pembelajaran: handling dynamic element

Dynamic element adalah elemen UI yang atribut, posisi, timing kemunculan, atau state-nya dapat berubah antar refresh/session. Pada automation, masalah utama biasanya bukan “elemen tidak ada”, tetapi “elemen belum siap atau locator terlalu rapuh”.

Prinsip implementasi yang dipelajari

- Gunakan locator yang stabil dan bermakna, misalnya data-testid, id yang memang stabil, role/label, atau kombinasi atribut yang relatif permanen.
- Utamakan explicit wait berbasis kondisi (visible, clickable, presence, attribute/state) daripada fixed sleep. Waiting harus mengikuti kondisi UI, bukan sekadar menunggu waktu tertentu.
- Untuk daftar/card dinamis, cari elemen berdasarkan konteks container atau text/attribute yang relevan, lalu ambil child element yang tepat agar selector tidak bergantung pada index yang mudah berubah.
- Bangun helper/wrapper reusable untuk wait dan interaction sehingga test case tetap ringkas dan aturan handling dynamic element konsisten.

Insight pembelajaran

Handling dynamic element bukan sekadar menambah timeout. Kualitas automation ditentukan oleh kombinasi locator yang stabil, wait yang tepat sasaran, dan sinkronisasi dengan state UI.

3. Pembelajaran: parallel execution

Parallel execution menjalankan beberapa test secara bersamaan untuk mengurangi total elapsed time. Pada framework berbasis Gradle/JUnit/TestNG, paralelisme harus dirancang bersama lifecycle driver, test data, dan resource agar kecepatan tidak mengorbankan isolation.

Konsep utama yang perlu diperhatikan

- Test harus independent: satu test tidak boleh bergantung pada state atau urutan test lain.
- WebDriver harus isolated per thread/test. Hindari satu instance driver global yang dipakai banyak test secara bersamaan.
- Test data harus aman terhadap collision. Gunakan akun/data unik atau mekanisme data preparation yang thread-safe.
- External resource seperti file download, screenshot, port, atau folder output perlu diberi nama unik per test/thread.
- Parallelism perlu diseimbangkan dengan kapasitas environment; menambah thread tanpa batas dapat meningkatkan contention dan justru memperpanjang runtime.

4. Review implementasi dari test report

Area	Observasi	Review / tindak lanjut
Stability	6 tests, 0 failures, 0 skipped	Positif. Uji ulang beberapa kali untuk memeriksa flaky behavior.
Coverage	Tiga child test terlihat pada report	Struktur test sudah terkelompok. Perlu pastikan skenario dynamic element tersebar pada positive path.
Execution time	Total sekitar 1m15s	Masih ada peluang optimasi melalui parallel execution, terutama bila test sleep dan scroll dioptimalkan.
Isolation	Tidak dapat dibuktikan hanya dari report	Perlu verifikasi driver lifecycle, test data, dan file/output agar aman saat paralel.

5. Pembelajaran yang dapat dibawa ke implementasi berikutnya

- Stabilitas test lebih banyak ditentukan oleh sinkronisasi dan locator daripada jumlah retry.
- Dynamic element handling sebaiknya dibuat sebagai reusable utility/pattern, bukan solusi ad-hoc di setiap test case.
- Parallel execution harus dimulai setelah test isolation cukup matang; paralelisasi terhadap test yang saling berbagi state hanya mempercepat munculnya race condition.

6. Rekomendasi implementasi praktis

A. Standardisasi locator: Tetapkan prioritas locator: data-testid/id stabil → role/label → CSS/XPath berbasis atribut bermakna. Hindari absolute XPath dan index selector bila tidak diperlukan.

B. Aktifkan parallel secara bertahap: Mulai 2 worker, ukur durasi dan failure rate, lalu naikan hanya bila environment stabil.

C. Observability: Simpan screenshot dan log dengan nama unik per test/thread agar debugging parallel failure tetap mudah.

7. Kesimpulan

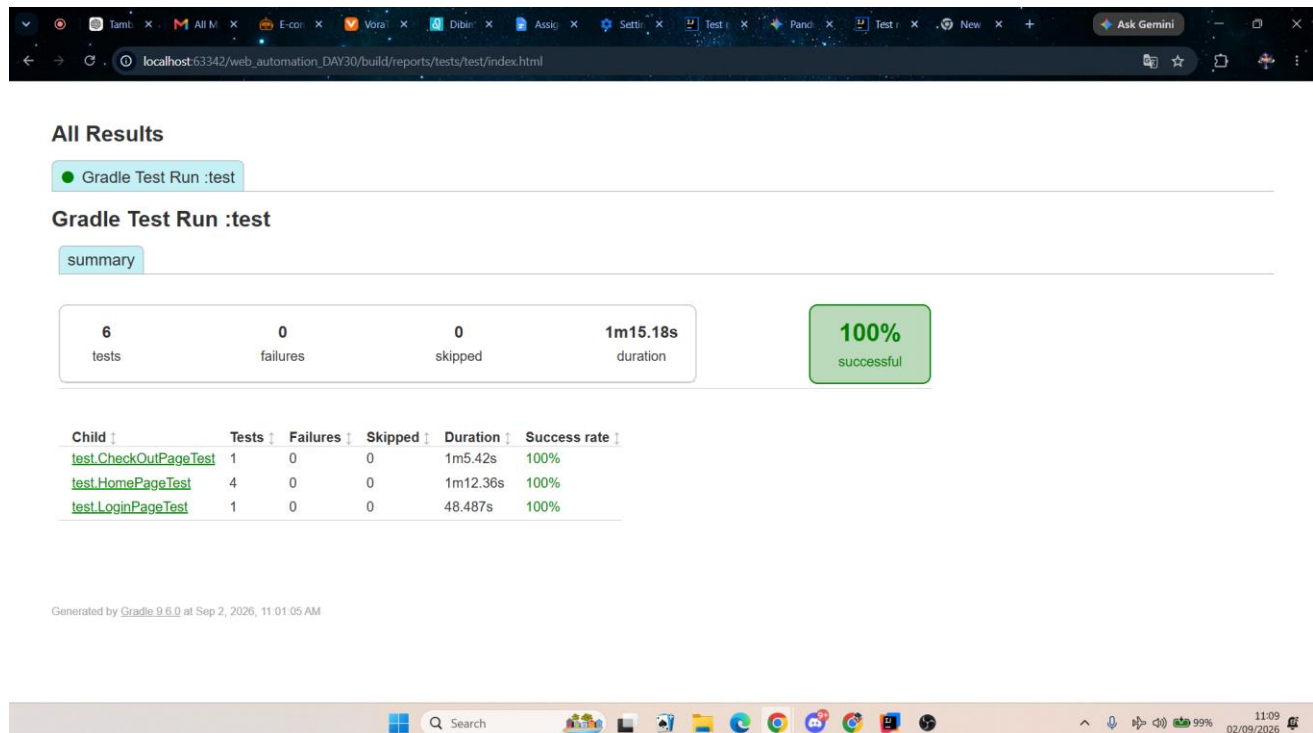
Secara umum, hasil eksekusi pada report menunjukkan implementasi berjalan dengan baik (100% successful). Pembelajaran paling penting dari handling dynamic element adalah membuat automation menunggu kondisi UI yang benar dan menggunakan locator yang tahan perubahan. Sementara itu, parallel execution memberikan peluang pengurangan waktu eksekusi, tetapi efektivitasnya sangat bergantung pada independensi test, isolasi driver/data, dan kapasitas environment.

Kesimpulan review

Fondasi sudah baik dari sisi hasil eksekusi. Tahap peningkatan berikutnya sebaiknya fokus pada pembuktian reliability (repeated run), audit explicit wait/locator, kemudian menerapkan parallel execution secara bertahap sambil memonitor failure rate dan total duration.

Lampiran: bukti test report

Screenshot yang menjadi dasar ringkasan hasil eksekusi:



Gambar 1. Gradle Test Report - 6 tests, 0 failures, 0 skipped, 100% successful.

Dynamic Element Handling • Parallel Execution • Gradle Test Report